Week 1 Study Guide: Spine, Skills, Experiment

Week of November 2–7, 2026 · 12 hours · 6 sessions

This week starts all three layers of the plan:

- **Spine:** Axler, *Linear Algebra Done Right*, chapter 1
- **Skills:** Harvard Stat 110, lectures 1–4
- **Experiment:** a Benford's law test

Module 1 keeps going on the side: Saturday's experiment gets committed to your repo, which is Module 1 session 2 practice anyway.

Contents

1. This week at a glance
2. Spine: Axler chapter 1
3. Skills: Stat 110, lectures 1–4
4. Experiment: Benford's law test
5. Key takeaways
6. How this week connects to your career paths
7. Answers

This week at a glance

Day	Layer	Session	Material
Mon	Spine	1. Fields and $\mathbb{F}^n$	Axler 1A
Tue	Skills	1. Counting	Stat 110 lectures 1–2
Wed	Spine	2. Vector spaces	Axler 1B
Thu	Skills	2. Birthdays and conditional probability	Stat 110 lectures 3–4
Fri	Spine	3. Subspaces and direct sums	Axler 1C
Sat	Experiment	Benford's law test	Code below
Sun	Rest		

Spine: Axler chapter 1

Chapter 1 sets up the objects everything else acts on: vector spaces and their subspaces. Read with a pencil. Axler's proofs are short, and redoing them yourself is the point.

Monday — Session 1: Fields and $\mathbb{F}^n$ (Axler 1A)

Key ideas

- $\mathbb{F}$ stands for either $\mathbb{R}$ or $\mathbb{C}$.
- A list of length n with entries from $\mathbb{F}$ is a vector in $\mathbb{F}^n$:

$$x = (x_1, x_2, ..., x_n), \qquad x_j \in \mathbb{F}$$

- Addition and scalar multiplication work coordinate by coordinate:

$$x + y = \left(x_1 + y_1, ..., x_n + y_n\right), \qquad \lambda x = (\lambda x_1, ..., \lambda x_n)$$

- The field properties (commutativity, associativity, identities, inverses) carry over from $\mathbb{F}$ to $\mathbb{F}^n$.

Worked example: the square roots of i

Write $z = a + bi$. Then

$$z^2 = \left(a^2 - b^2\right) + 2ab\,i = i,$$

so $a^2 - b^2 = 0$ and $2ab = 1$. That gives $a = b = \pm\frac{1}{\sqrt{2}}$, and the two roots are

$$z = \pm\frac{1 + i}{\sqrt{2}}.$$

Tasks

- [ ] Read 1A, redoing each proof before reading Axler's
- [ ] Do 5 exercises from the end of 1A
- [ ] Write 3 lines in `NOTES.md`: what $\mathbb{F}^n$ is, and one thing that surprised you

Wednesday — Session 2: Vector spaces (Axler 1B)

Key ideas

- A vector space is any set V with an addition and a scalar multiplication that satisfy eight rules: commutativity, associativity, additive identity, additive inverse, multiplicative identity, and two distributive laws.
- Vectors don't have to be lists. Functions and polynomials qualify too.
- The example to remember is $\mathbb{F}^S$, the set of all functions from a set S to $\mathbb{F}$:

$$\mathbb{F}^S = \{f: S \to \mathbb{F}\}, \qquad (f + g)(x) = f(x) + g(x), \qquad (\lambda f)(x) = \lambda f(x)$$

- A week of daily spending is a function from $S = \{\text{Mon}, ..., \text{Sun}\}$ to $\mathbb{R}$, so it is a vector in $\mathbb{R}^S$. Since S has 7 elements, this is really $\mathbb{R}^7$.

Worked example: prove $0v = 0$ for every vector v

By distributivity,

$$0v = (0 + 0)\, v = 0v + 0v.$$

Add the additive inverse of $0v$ to both sides, and you get $0 = 0v$. ■

Tasks

- [ ] Read 1B, and redo the proofs that the additive identity and additive inverses are unique
- [ ] Prove $a\,0 = 0$ for every scalar a, in the style of the worked example
- [ ] Do 4 exercises from the end of 1B

Friday — Session 3: Subspaces and direct sums (Axler 1C)

Key ideas

- A subset $U \subseteq V$ is a **subspace** if it contains 0 and is closed under addition and scalar multiplication.
- The **sum** of two subspaces is

$$U + W = \{\, u + w : u \in U,\ w \in W \,\}.$$

- The sum is **direct**, written $U \oplus W$, when every vector in $U + W$ splits into a U-part and a W-part in exactly one way.
- For two subspaces, there's a quick test:

$$U + W \text{ is direct} \Leftrightarrow U \cap W = \{0\}.$$

Worked example, with a fraud link

In $\mathbb{R}^7$, let

$$U = \{\, (c, c, ..., c) : c \in \mathbb{R} \,\}, \qquad W = \{\, x \in \mathbb{R}^7 : \sum_{j=1}^{7} x_j = 0 \,\}.$$

U is the constant weeks and W is the weeks that sum to zero. A constant vector that sums to zero has $7c = 0$, so $c = 0$ and $U \cap W = \{0\}$. Every week x splits as

$$x = \underbrace{(\bar{x}, ..., \bar{x})}_{\in U} + \underbrace{(x_1 - \bar{x}, ..., x_7 - \bar{x})}_{\in W},$$

where $\bar{x}$ is the average. So $\mathbb{R}^7 = U \oplus W$. The W-part, the deviation from average, is exactly what an anomaly check looks at.

```
import numpy as np

week = np.array([120., 95., 130., 110., 480., 105., 100.])  # daily spend

avg = np.full(7, week.mean())  # the U-part: a constant vector
dev = week - avg               # the W-part: sums to zero

print(dev.round(1))                  # Friday's +317.1 stands out
print(dev.sum().round(10))           # 0.0, so dev is in W
print(np.allclose(avg + dev, week))  # True: the split rebuilds the week
```

Tasks

- [ ] Read 1C
- [ ] Run the code, and check by hand that U and W are subspaces
- [ ] Do 4 exercises from the end of 1C

Spine quiz

1. Is $\{ (x, y, z) \in \mathbb{R}^3 : x + 2y + 3z = 0 \}$ a subspace of $\mathbb{R}^3$? What if the right side is 4?
2. Is $\{ (x, y) \in \mathbb{R}^2 : xy = 0 \}$ a subspace of $\mathbb{R}^2$?
3. Are the polynomials of degree exactly 3 a subspace of $\mathcal{P} (\mathbb{R})$?
4. In $\mathbb{R}^3$, let $U = \{ (x, x, x) \}$ and $W = \{ (x, y, z) : x + y + z = 0 \}$. Is $U + W$ a direct sum?
5. You have three subspaces, and each pair of them intersects only in $\{0\}$. Must their sum be direct?

Skills: Stat 110, lectures 1–4

The lectures are free on the Stat 110 site and on YouTube, about 50 minutes each. Watch at 1.25× if that suits you, and pause to try each example before Blitzstein solves it.

Tuesday — Session 1: Counting (lectures 1–2)

Key ideas

- With equally likely outcomes:

$$P (A) = \frac{\text{number of outcomes in } A}{\text{number of all outcomes}}$$

- The **multiplication rule** counts choices made in steps: n_1 ways, then n_2 ways, gives $n_1 \cdot n_2$ in total.
- Order matters for arrangements and not for groups, which is where "n choose k" comes from:

$$\binom{n}{k} = \frac{n!}{k! \, (n - k) \, !}$$

- A **story proof** explains an identity by counting the same thing two ways.

Worked example: PINs

A 4-digit PIN has $10^4 = 10,000$ options. If no digit may repeat, there are $10 \cdot 9 \cdot 8 \cdot 7 = 5,040$. That one rule cuts an attacker's search almost in half.

Tasks

- ☐ Watch lectures 1 and 2
- ☐ Explain in your own words why $\binom{n}{k} = \binom{n}{n-k}$
- ☐ Count: how many 8-character passwords use only lowercase letters? Lowercase letters plus digits?

Thursday — Session 2: Birthdays and conditional probability (lectures 3–4)

Key ideas

- The **birthday problem**: matches show up far sooner than intuition says. With n people and 365 days,

$$P(\text{ at least one match }) = 1 - \prod_{k=0}^{n-1} \frac{365-k}{365}.$$

This is the same math behind hash collisions in cryptography.

- **Conditional probability** is the chance of A once B is known:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

- **Bayes' rule** flips the condition:

$$P(A \mid B) = \frac{P(B \mid A)\ P(A)}{P(B)}$$

Worked example: the base-rate trap

0.1% of transactions are fraud. An alert catches 99% of fraud, and wrongly fires on 1% of normal transactions. What fraction of alerts are real fraud?

$$P(\text{ fraud} \mid \text{alert}) = \frac{0.99 \times 0.001}{0.99 \times 0.001 + 0.01 \times 0.999} = \frac{0.00099}{0.01098} \approx 9\%.$$

Most alerts are false alarms, even from a "99% accurate" model.

```
import numpy as np

rng = np.random.default_rng(0)

def shared_birthday(n, trials=100_000):
    days = rng.integers(0, 365, size=(trials, n))
    days.sort(axis=1)
    return (np.diff(days, axis=1) == 0).any(axis=1).mean()

exact = 1 - np.prod((365 - np.arange(23)) / 365)
print(shared_birthday(23), exact)  # both about 0.507
```

Tasks

- [ ] Watch lectures 3 and 4
- [ ] Run the simulation, then plot the match chance for $n = 2$ to 60
- [ ] Redo the base-rate example with fraud at 1% instead of 0.1%

Skills quiz

1. How many people do you need for a better-than-even chance that two share a birthday?
2. A hash function has 2^{32} possible outputs. Roughly how many random inputs until a collision is a coin flip?
3. In the base-rate example, what is $P(\text{ alert} \mid \text{fraud })$? What is $P(\text{ fraud} \mid \text{alert })$? Why do they differ so much?
4. Two dice are rolled, and at least one shows a 6. What's the chance both are 6?

Experiment: Benford's law test

Saturday, 2 hours. This is the first slice of the fraud experiment.

In many natural datasets, the leading digit is 1 about 30% of the time, and 9 less than 5% of the time. Made-up numbers usually don't follow this pattern, which makes it a classic first check in forensic accounting.

$$P(\text{ leading digit} = d) = \log_{10}\left(1 + \frac{1}{d}\right), \qquad d = 1, 2, ..., 9$$

Leading digit d	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Benford share	30.1%	17.6%	12.5%	9.7%	7.9%	6.7%	5.8%	5.1%	4.6%

The code compares three made-up datasets against Benford:

- **Natural:** amounts spanning several orders of magnitude
- **Fabricated:** amounts invented evenly between $100 and $999

- **Narrow:** real-looking amounts that sit in a tight range

It scores each one with a chi-square statistic,

$$\chi^2 = \sum_{d=1}^{9} \frac{n\left(\widehat{p}_d - p_d\right)^2}{p_d},$$

where $\widehat{p}_d$ is the observed share of digit d, p_d is Benford's share, and n is the number of amounts.

```
import numpy as np

rng = np.random.default_rng(0)
benford = np.log10(1 + 1 / np.arange(1, 10))

def first_digit(x):
    x = np.asarray(x, dtype=float)
    return (x / 10 ** np.floor(np.log10(x))).astype(int)

def digit_shares(x):
    return np.bincount(first_digit(x), minlength=10)[1:] / len(x)

datasets = {
    "natural": rng.lognormal(mean=6, sigma=1.5, size=5000),
    "fabricated": rng.uniform(100, 999, size=500).round(2),
    "narrow": rng.lognormal(mean=6, sigma=0.5, size=5000),
}

for name, x in datasets.items():
    shares = digit_shares(x)
    chi2 = (len(x) * (shares - benford) ** 2 / benford).sum()
    mad = np.abs(shares - benford).mean()
    print(f"{name:>10}  chi2={chi2:8.1f}  MAD={mad:.4f}")

# A chi-square above 15.5 (8 degrees of freedom) rejects Benford at the 5% level
```

Tasks

- [ ] Create a branch `experiment/benford`, and save the code as `notebooks/02_benford.ipynb`
- [ ] Plot each dataset's digit shares as bars, with Benford as a line
- [ ] Note which datasets fail, and why "narrow" fails even though nothing is fake
- [ ] Commit, push, and open a pull request (Module 1 practice)
- [ ] At work, **not** in the public repo: run the same test on one expense or invoice table

Experiment quiz

1. What share of numbers should start with 1? With 9?
2. Why does Benford fail for amounts that all fall between $100 and $500?
3. A vendor's invoices fail the test. Is that proof of fraud? What would you check next?
4. Why does the test need hundreds of numbers rather than 20?

Key takeaways

1. **A vector is anything you can add and scale.** Lists, functions, polynomials and weeks of spending all qualify. Once you see data as vectors in $\mathbb{F}^n$ or $\mathbb{F}^S$, all of linear algebra applies to it.
2. **Subspaces are the "allowed shapes" inside a space.** The test is short: contains 0, closed under addition, closed under scaling. A line or plane not through the origin fails.
3. **A direct sum splits every vector uniquely.** $\mathbb{R}^7 = U \oplus W$ breaks a week into "average" plus "deviation." Splitting data into "normal part" plus "what's left" is the core idea of anomaly detection.
4. **For two subspaces, $U \cap W = \{0\}$ means direct. For three or more, checking pairs isn't enough.** (Quiz question 5.)
5. **Counting sets the size of a search.** Every password, PIN or key length is a counting problem: 26^8 vs 36^8 is the difference between a fast and a slow brute-force attack.
6. **Collisions come at the square root.** With N possible values, a repeat is likely after about $\sqrt{N}$ tries. That's 23 people for 365 birthdays, and about 2^{16} inputs for a 2^{32}-output hash.
7. $P(A \mid B) \neq P(B \mid A)$**.** When the thing you're looking for is rare, even an accurate detector produces mostly false alarms. Always ask for the base rate.
8. **Every test has assumptions.** Benford works only on data spanning several orders of magnitude. A failure is a reason to look closer, never proof on its own.

How this week connects to your career paths

This week's idea	Fraud / risk analyst	Security data scientist	AI red teaming	Cryptography
Data as vectors ($\mathbb{F}^n$, $\mathbb{F}^S$)	Each vendor's monthly spend is a vector you can compare	Each user's login pattern becomes a feature vector for a model	Every word an LLM reads becomes a vector (an embedding) in $\mathbb{R}^d$	Messages and keys are vectors over finite fields such as $\mathbb{F}_2^n$
Direct sums ($U \oplus W$)	"Average + deviation" is the basis of the rolling z-score check	Later, PCA splits data into "normal directions" plus a residual that becomes the anomaly score	A known jailbreak removes the "refusal direction" from a model: it splits activations into a part along one direction plus the rest, then drops that part	Error-correcting codes split a space into valid codewords plus correctable errors
Counting	How many invoice-number formats could a fraudster guess?	Size of the space an attacker searches when password spraying	Size of the space an automated jailbreak search must explore	Key length decides whether brute force is hopeless: 2^{128} is out of reach

This week's idea	Fraud / risk analyst	Security data scientist	AI red teaming	Cryptography
Birthday bound ($\sqrt{N}$)	Duplicate invoice numbers appear sooner than you'd expect by chance	Hash-based deduplication of logs has a collision risk you can calculate	Collisions in hashed or bucketed content filters	Why a 256-bit hash gives only 128-bit collision resistance
Base rates and Bayes	Most fraud alerts are false; tune thresholds to an analyst's daily capacity	Alert fatigue in a security operations center is a base-rate problem	A guardrail that blocks 1% of harmless prompts annoys millions of users; measure both error types	Estimating the chance an attack succeeds after observing partial information
Benford's law	A standard forensic accounting check, used by auditors and fraud examiners	A model of "natural" data you can test log volumes or byte counts against	Testing whether model-generated numbers look natural or invented	—

One example worth remembering from each column:

- **Fraud:** You run Benford on a vendor's invoices and it fails. Base rates say: don't accuse; pull the 20 invoices with the most over-represented leading digit and look for duplicates and amounts just under approval limits.
- **Security data science:** A login model flags 1% of logins. With 2 million logins a day, that's 20,000 alerts. If real attacks are 1 in 100,000, nearly all of those alerts are noise. This week's Bayes calculation tells you how much better the model must get.
- **AI red teaming:** Removing a refusal direction is a projection: split each activation a into its part along a unit vector r and the rest, $a = (a \cdot r)\, r + (a - (a \cdot r)\, r)$, then keep only the rest. That's the same idea as $U \oplus W$, which you'll formalize in Axler chapter 6 (orthogonal complements).
- **Cryptography:** A 64-bit hash looks safe because 2^{64} is huge, but by the birthday bound a collision is likely after about $2^{32} \approx 4$ billion tries. A laptop can manage that. This is why modern hashes are 256 bits.

Answers

Spine

1. **Yes.** It contains 0 and is closed under addition and scaling. With 4 on the right side, **no**: $(0, 0, 0)$ isn't in it.
2. **No.** $(1, 0)$ and $(0, 1)$ are both in it, but their sum $(1, 1)$ isn't, since $1 \cdot 1 \neq 0$.

3. **No.** The zero polynomial doesn't have degree 3. Also, $x^3 + \left(-x^3 + x \right) = x$ has degree 1, so it isn't closed under addition.
4. **Yes.** If $(x, x, x) \in W$ then $3x = 0$, so $x = 0$ and $U \cap W = \{0\}$. In fact $\mathbb{R}^3 = U \oplus W$, the 3-dimensional version of Friday's example.
5. **No.** Take three different lines through the origin in $\mathbb{R}^2$: the x-axis, the y-axis and the line $y = x$. Each pair meets only at 0, yet

$$(1,1) = (1,0) + (0,1) + (0,0) = (0,0) + (0,0) + (1,1)$$

splits two different ways. The pairwise test only works for two subspaces.

Skills

1. **23 people** gives a match chance of about 50.7%.
2. **About 77,000**, roughly $1.18 \times \sqrt{2^{32}} = 1.18 \times 2^{16}$. Collisions arrive around the square root of the number of outputs, which is why hash outputs need to be long.
3. $P(\text{alert} \mid \text{fraud}) = 99\%$, but $P(\text{fraud} \mid \text{alert}) \approx 9\%$. Fraud is so rare that the 1% of false alarms on the huge pool of normal transactions outnumbers the true catches.
4. $\frac{1}{11}$. "At least one 6" leaves 11 equally likely outcomes, and only $(6,6)$ has both.

Task answers

- Lowercase only: $26^8 \approx 2.1 \times 10^{11}$ passwords. With digits: $36^8 \approx 2.8 \times 10^{12}$, about 13 times more.
- With fraud at 1%:

$$P(\text{fraud} \mid \text{alert}) = \frac{0.99 \times 0.01}{0.99 \times 0.01 + 0.01 \times 0.99} = 50\%.$$

Experiment

Expected output: "natural" passes with a chi-square around 6. "Fabricated" (around 170) and "narrow" (around 1,500) fail badly.

1. $\log_{10} 2 \approx 30.1\%$ start with 1, and $\log_{10}(10/9) \approx 4.6\%$ start with 9.
2. Benford needs data spanning several orders of magnitude. Between $100 and $500, the leading digit can barely be anything but 1–4, so the range decides the digits, not honesty.
3. **No, it's a flag to investigate.** Next, check which digits are over-represented, duplicate invoices, round amounts, amounts just under approval limits, and the vendor itself.
4. Small samples swing a lot by chance. With 20 numbers you'd expect fewer than one leading 9, so the test can't tell noise from a real pattern. Aim for a few hundred numbers or more.